COMP0222 Simultaneous Localisation and Mapping

Lecture 11: Diving into ORB-SLAM2

Simon Julier

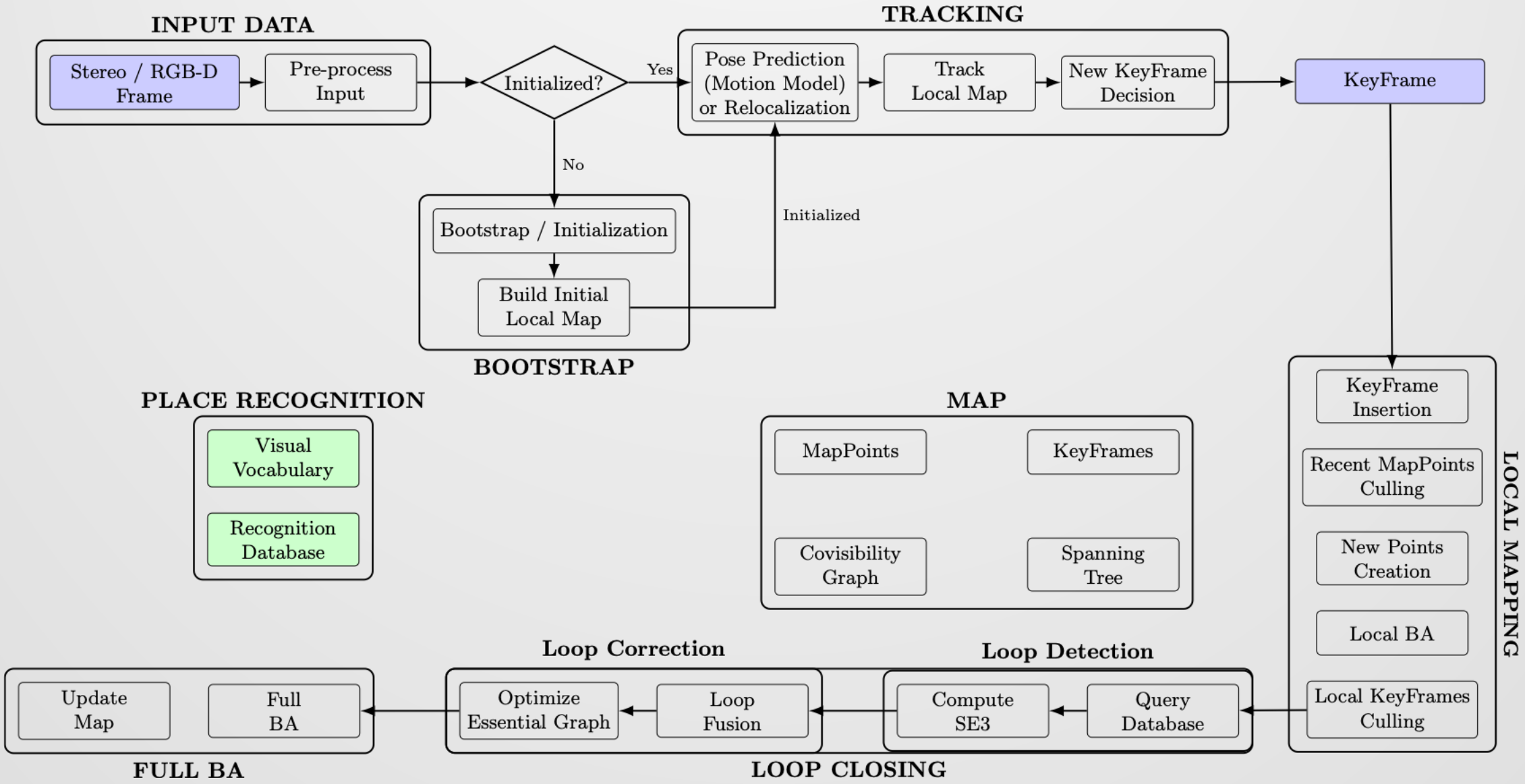
s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

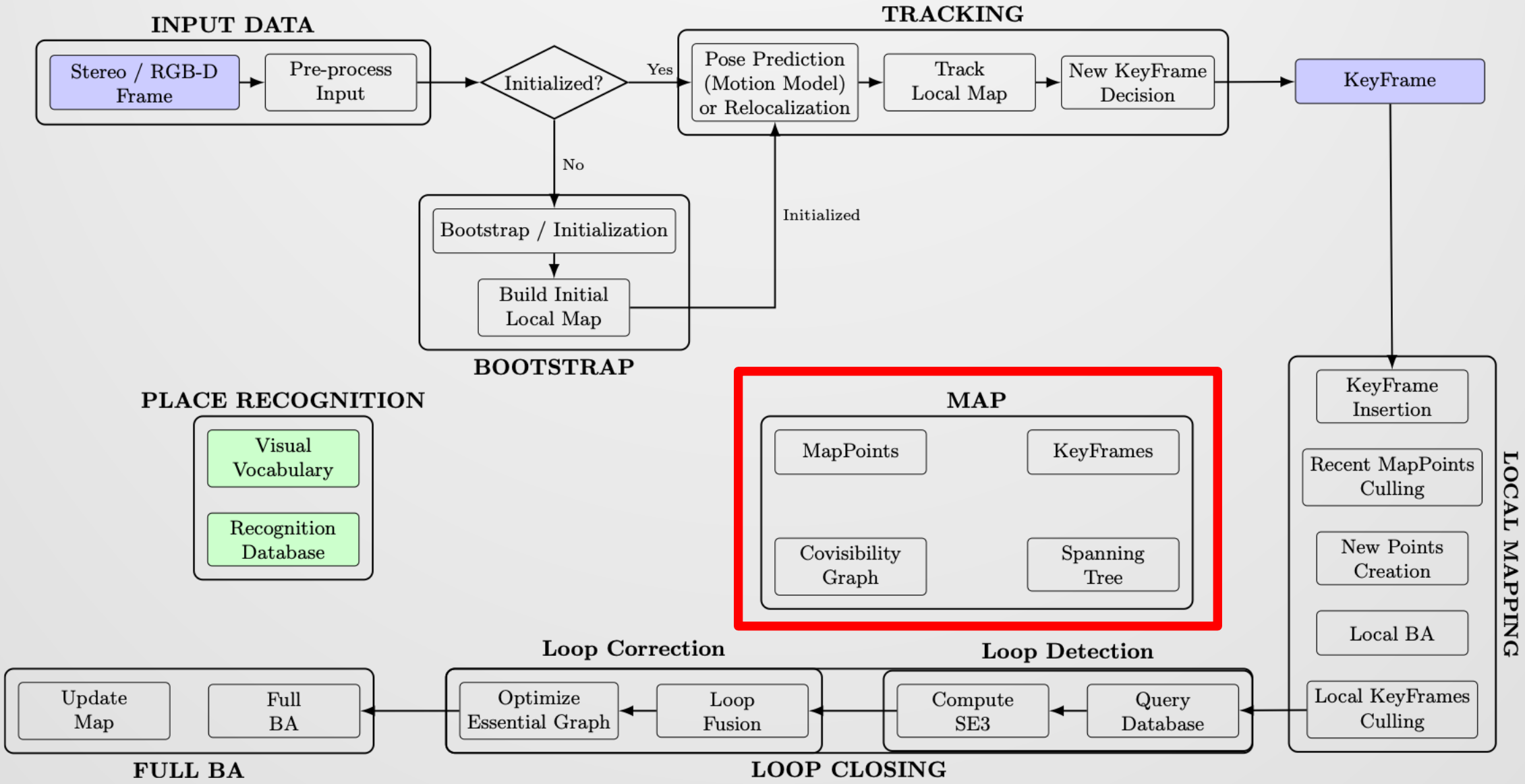
Motivation

- In the last lecture, we looked at COLMAP
- COLMAP is an SfM system designed towards producing the most accurate models in a reasonable amount of time
- ORB-SLAM2 is a SLAM system designed to localize, and to do so in real-time
- This means the architecture is broadly the same, but many detailed design choices are very different


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

Structure

- Map
- Place Recognition
- Input Data
- Bootstrap
- Tracking
- Local Mapping
- Loop Closing
- Full BA


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

ORB-SLAM Map Structure

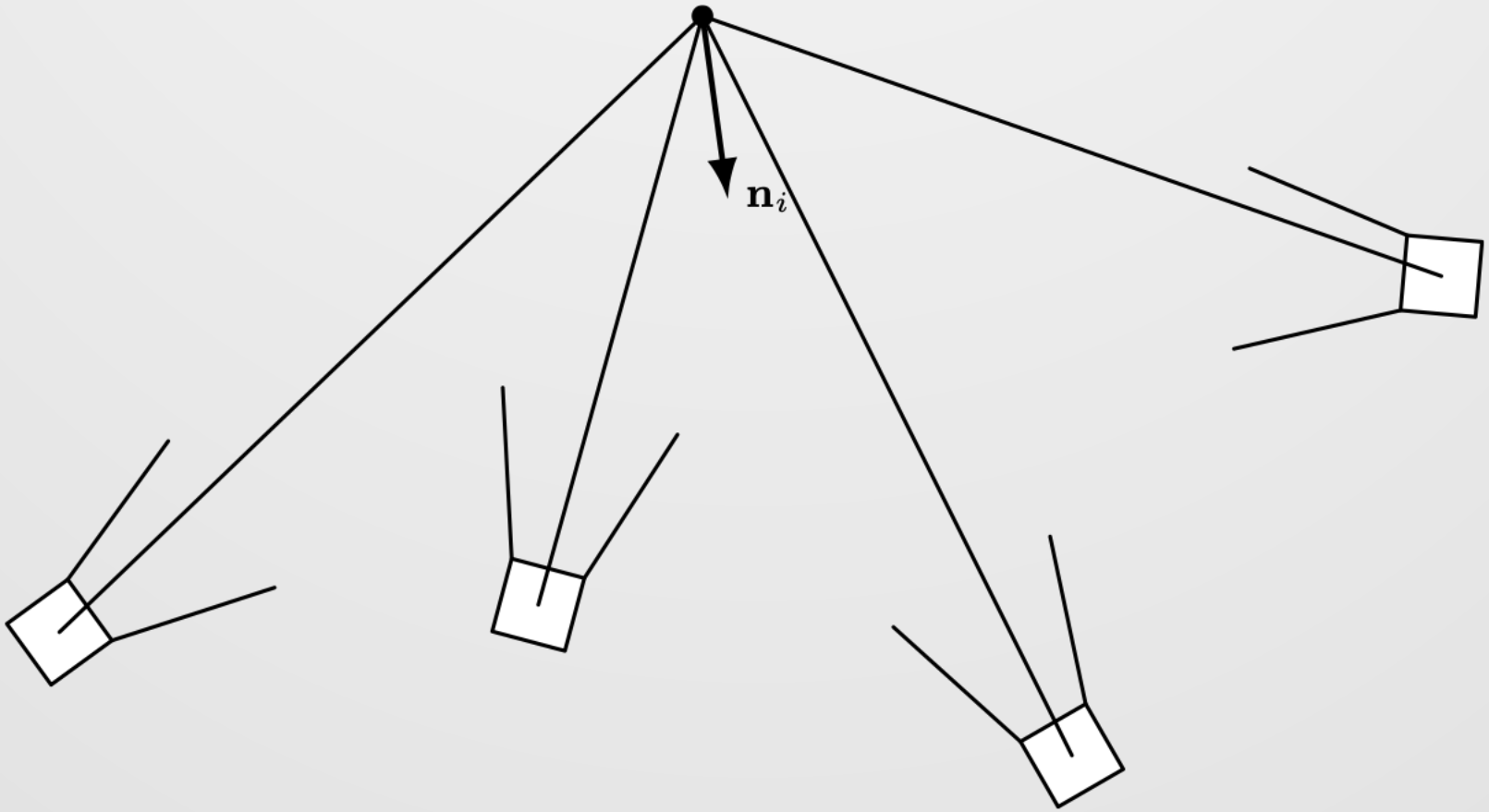
- ORB-SLAM uses a similar structure to COLMAP:
  - There is a collection of cameras and associated frames, known as keyframes
  - There is a collection of 3D points which correspond to the map points
- ORB-SLAM only maintains a subset of input frames as keyframes

Feature Points

- Each map point p_i stores:
  - The point 3D position in world coordinates $\mathbf{X}_{w,i}$
  - The ORB descriptor $\mathbf{D}_i$
  - The mean viewing direction $\mathbf{n}_i$
  - Minimium and maximum observable distances

- Map points are also called *keypoints*

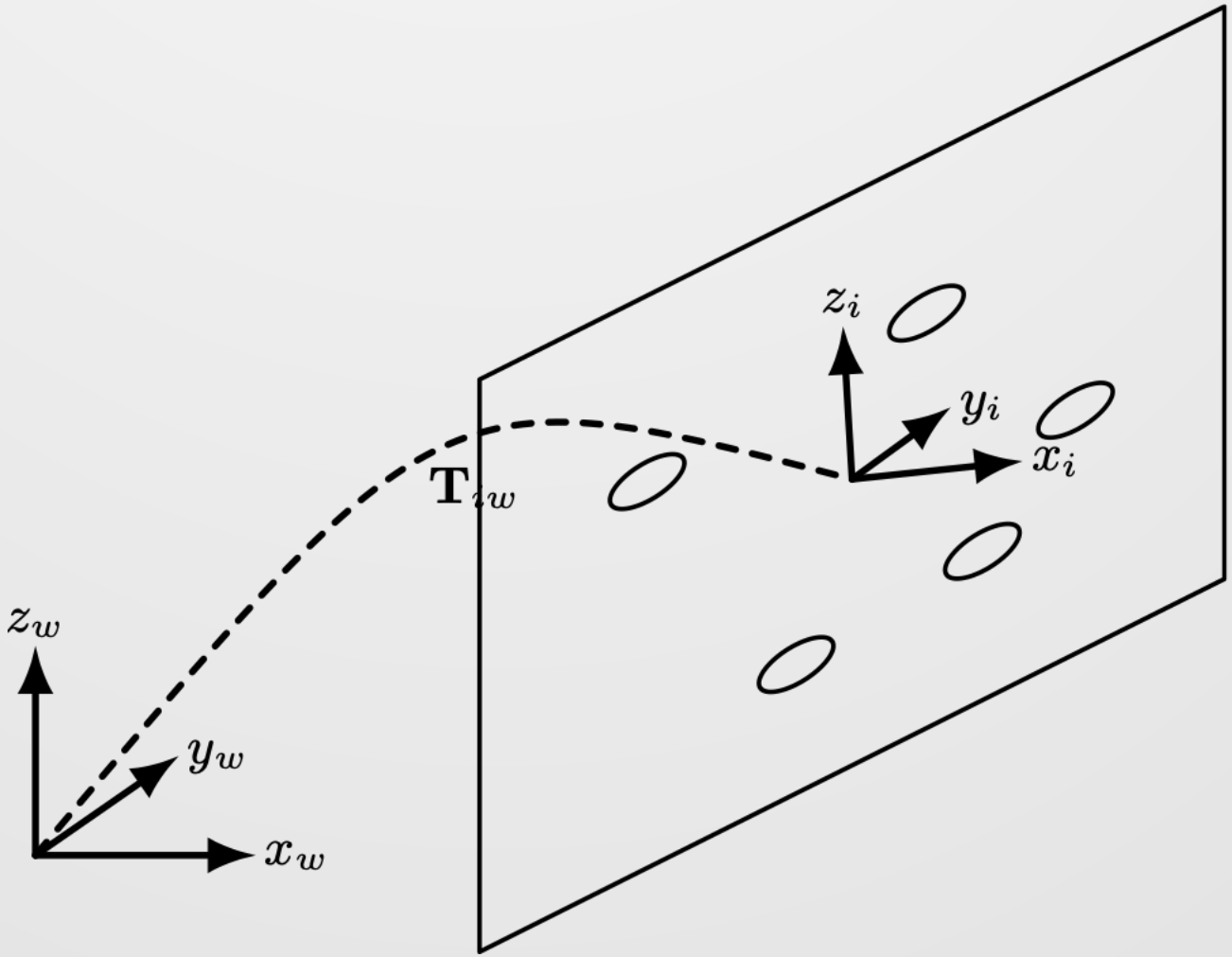
Key Point



Keyframes

- Each keyframe K_i stores:
  - The camera pose T_{iw}
  - The camera intrinsics
  - All the ORB features extracted from a frame, and whether they are connected to a map point

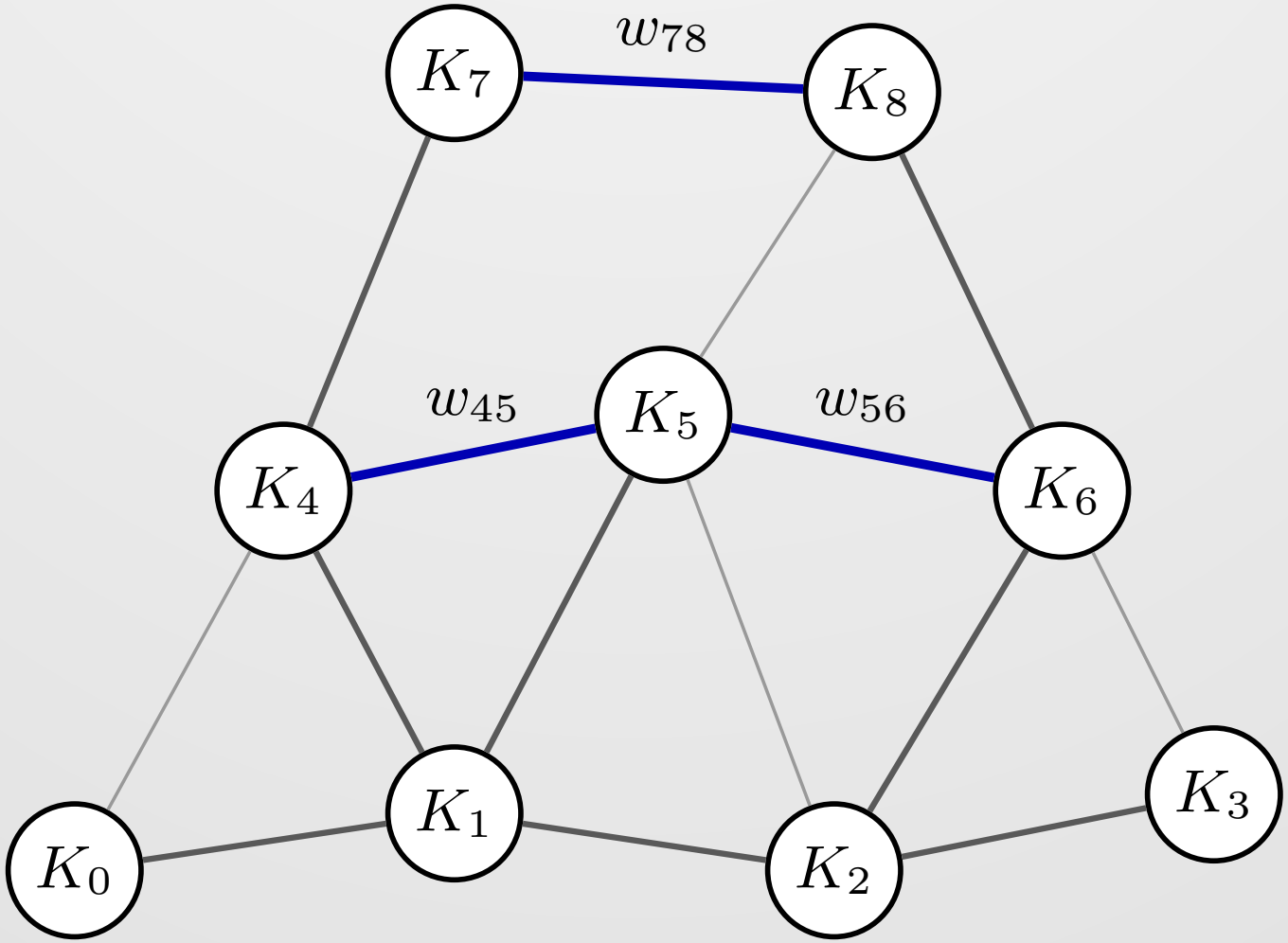
Keyframe



Covisibility Graph

- The Covisibility Graph is a bit like COLMAP's scenegraph
- It stores an edge between two keyframes if they share 15 or more map points
- The weight of an edge is equal to the number of shared points
- We'll see it's used later to do things like build local maps on the fly

Covisibilty Graph



Covisibility and Essential Graphs

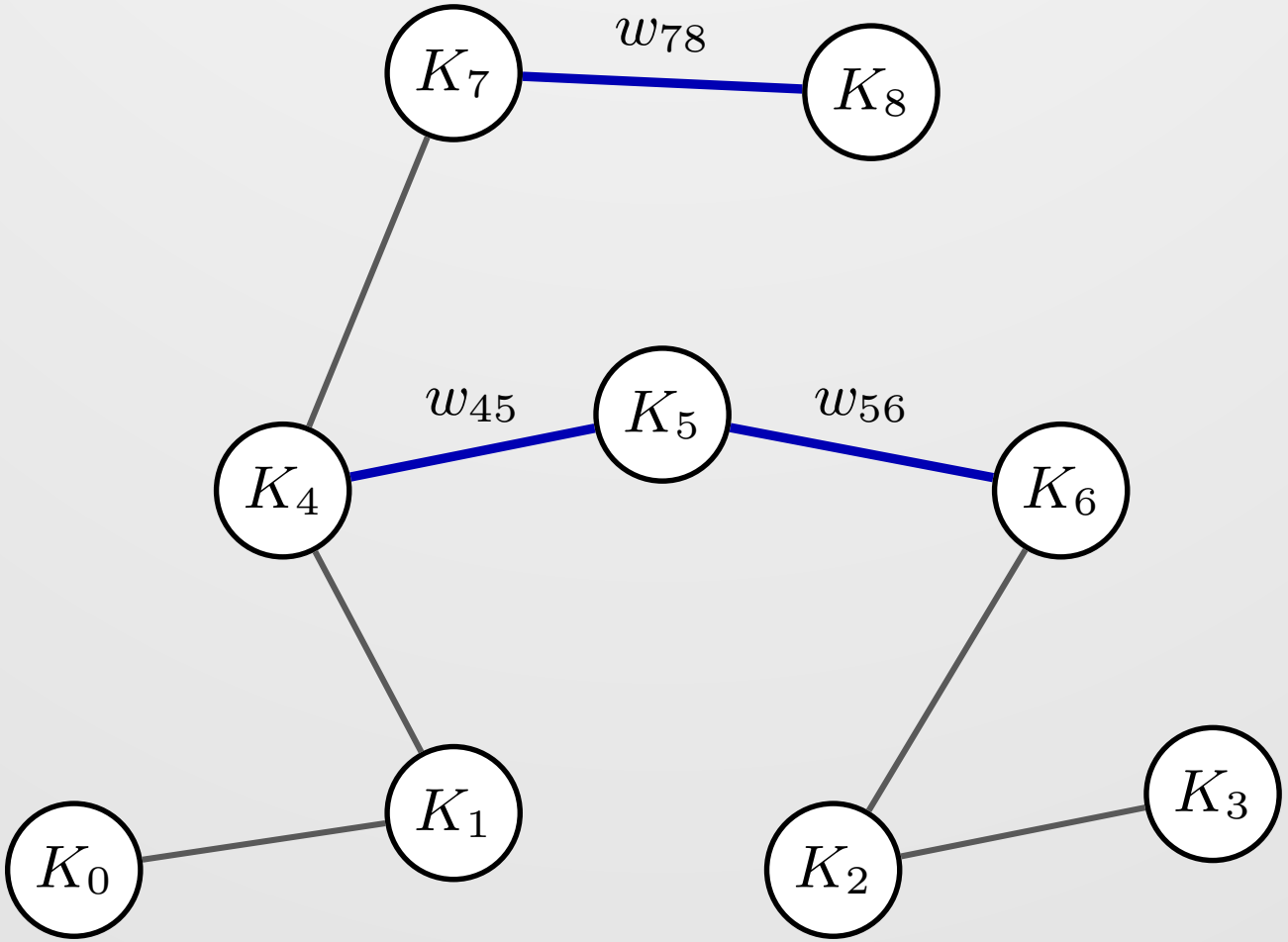
- Because the covisiblity graph stores connectivity between all the keyframes it can become very dense which can make operations very slow
- The Essential Graph is a lightweight version of it for more efficient computation for loop closure

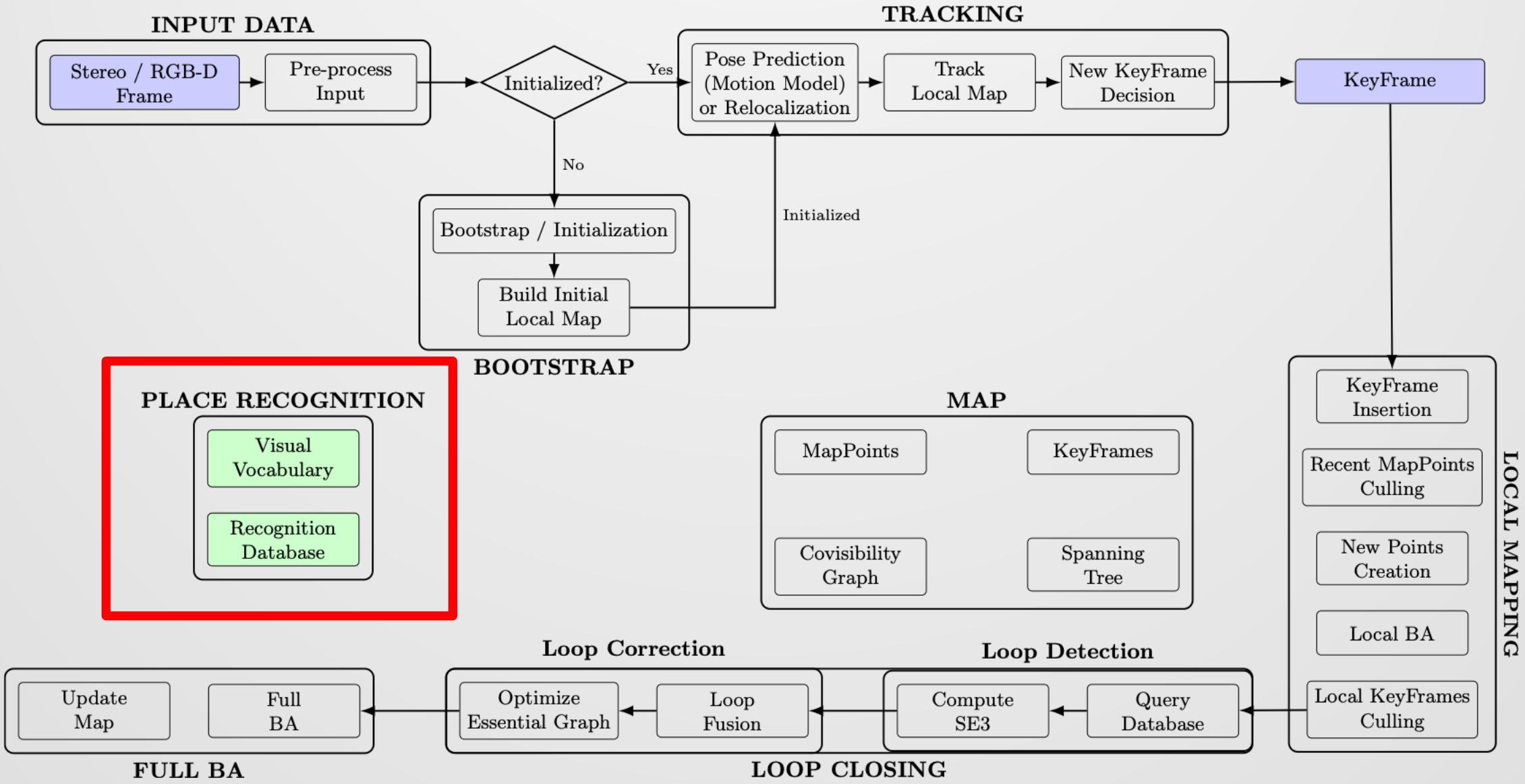
Essential Graph

- This is a spanning tree which connects all of the keyframes together using the smallest number of edges
- Edges are kept if there are at least 100 common features*
- The essential graph forms the factor graph which is used during loop closing

* However, then it isn't guaranteed to be a spanning tree…

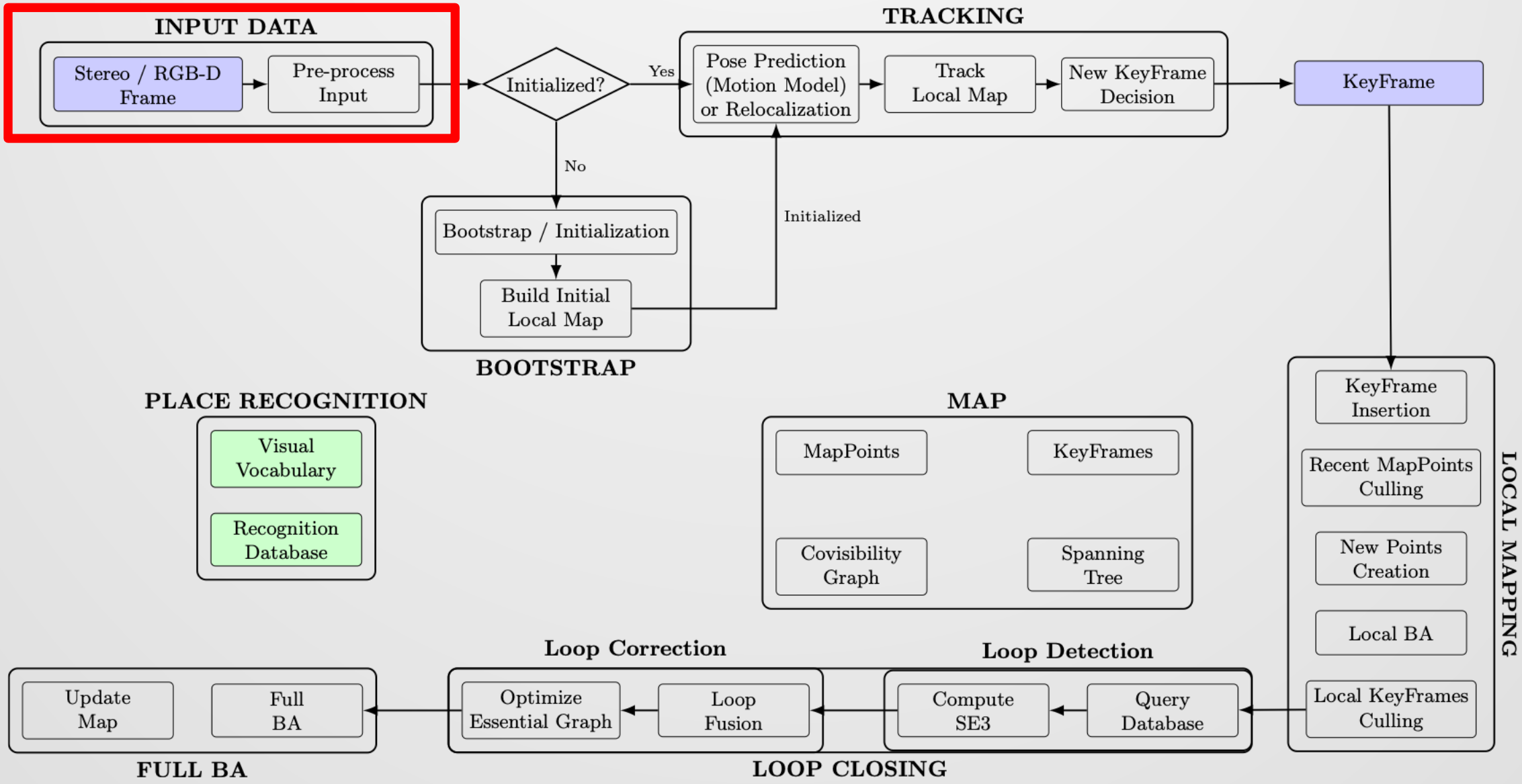
Essential Graph




INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

Place Recognition

- ORB-SLAM uses the DBoW2 bag of words library
- A generic dictionary is normally used which has been shown to give good average behaviour
- As each keyframe is analysed, an invert index is used to link each dictionary entry to the keyframe where it has been observed
- The dictionary can also be used to contrain ORB feature matches


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

Stereo / RGB-D Frame

- ORB-SLAM2 can take three types of camera input:
  - Monocular cameras
  - Stereo cameras
  - RGBD cameras

- Monocular cameras do not directly measure depth and so are *significantly harder* to deal with

Monocular Camera

- This is the single camera we saw with COLMAP
- It is assumed that the camera intrinsics *are known* through the whole run
- This typically means that you calibrate the camera separately, and then keep all the intrinsics constant for the duration of your SLAM run
- A similar feature extraction process to COLMAP is used

$$\mathbf{x}_m = (u_L, v_L)$$

ORB Feature Extraction

- ORB-SLAM introduces two refinements:
  - User-specified maximum number of ORB descriptors extracted per frame
  - Adaptive FAST descriptors thresholds
- FAST adaptation scheme:
  - In each grid cell the FAST detector looks for at least five corners
  - FAST thresholds are dynamically adjusted between user-specified limits

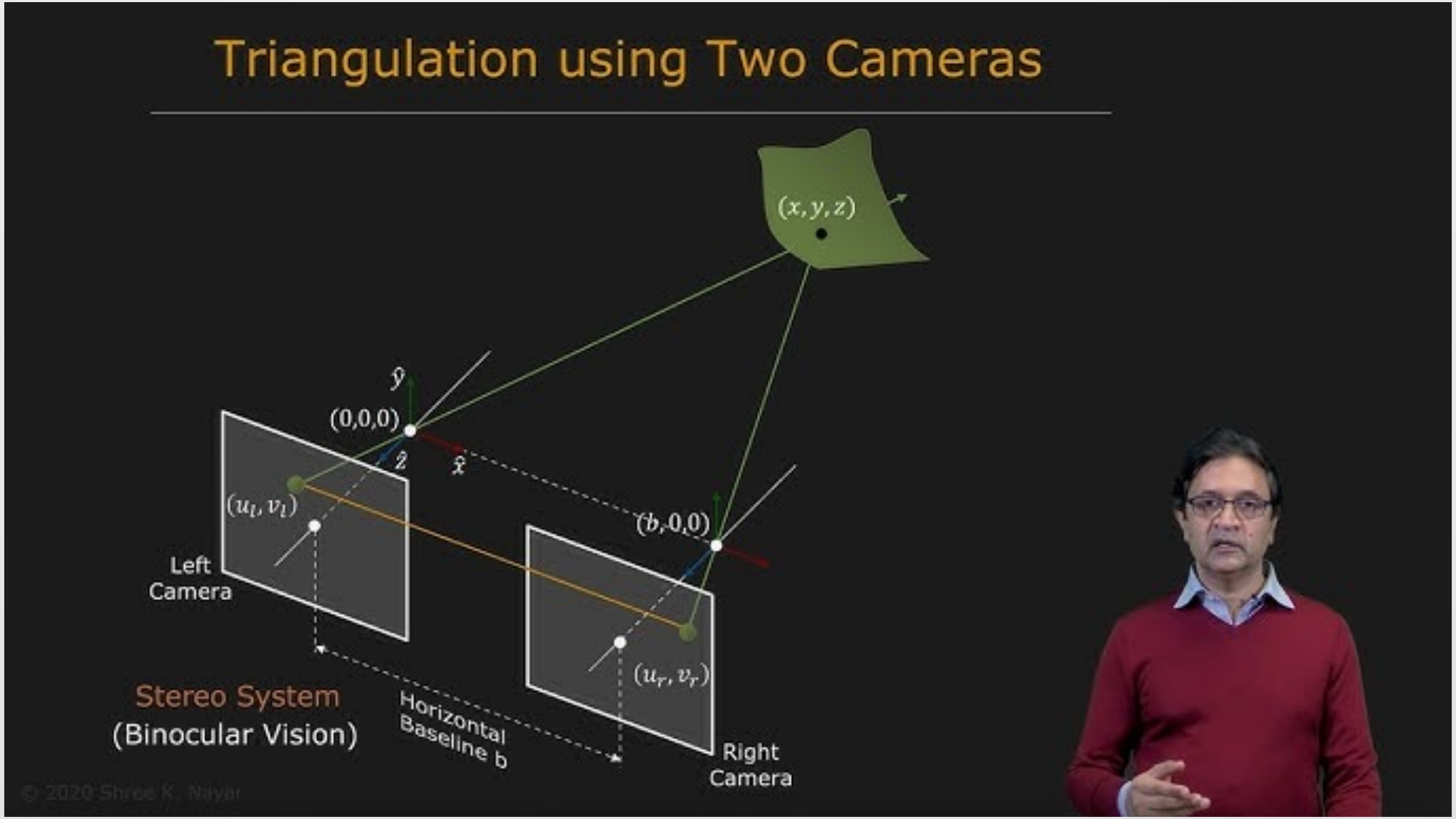
Stereo Camera

- This consists of two cameras offset from one another by a distance know as a baseline
- You can directly use triangulation to estimate the distance to points

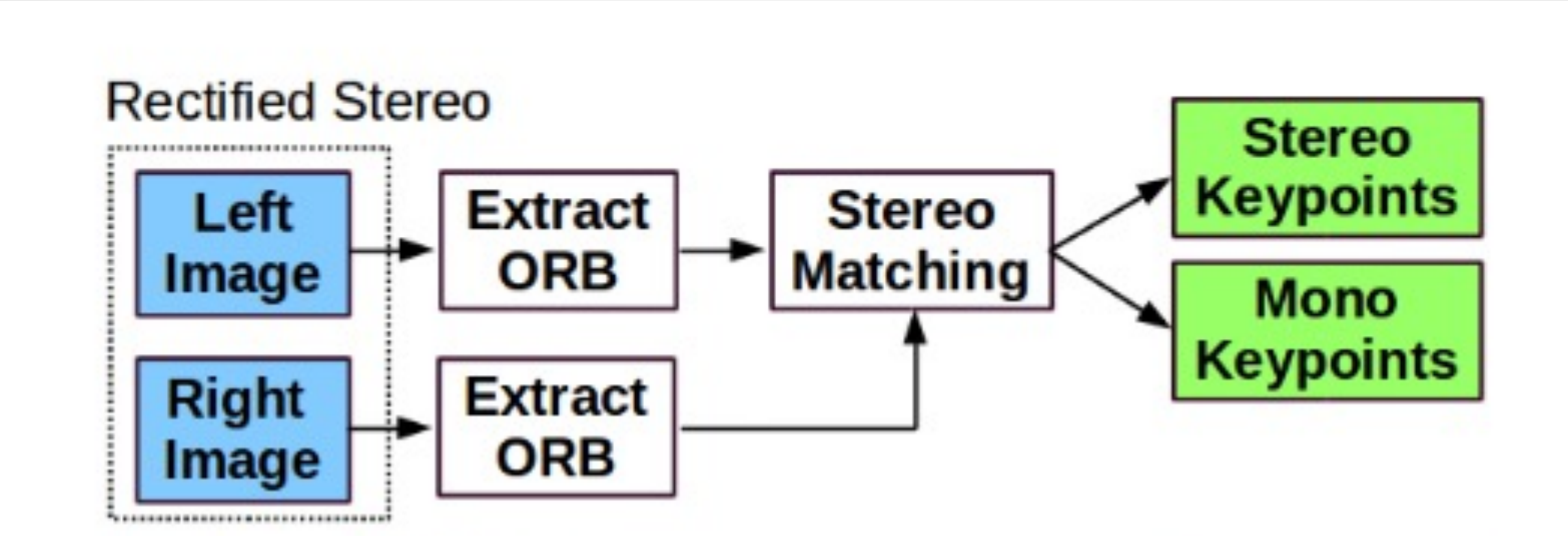
Example of a Stereo Camera

Stereo Camera

- This consists of two cameras offset from one another by a distance know as a baseline
- You can directly use triangulation to estimate the distance to points
- ORB-SLAM assumes the images are *stereo rectified*
- This means the only offset is on a horizontal scan line


Triangulation using Two Cameras
(x, y, z)
$\hat{y}$
(0,0,0)
$\hat{z}$
$\hat{x}$
(u_l, v_l)
(b,0,0)
Left Camera
(u_r, v_r)
Stereo System
(Binocular Vision)
Horizontal Baseline b
Right Camera
© 2020 Shree K. Nayar

Pre-Processing the Stereo Camera



Pre-Process Input

- Left frame and right frame ORB extractors are run in two separate threads
- Correspondences are identified by searching along the same scan line for features with similar ORB descriptors
- A correlation-based technique using templates is used to refine the match

Mono and Stereo Keypoints

- Mono keypoints can't be triangulated and so are just

$$\mathbf{x}_m = (u_L, v_L)$$

- Stereo keypoints are successfully triangulated and are observations of the form,

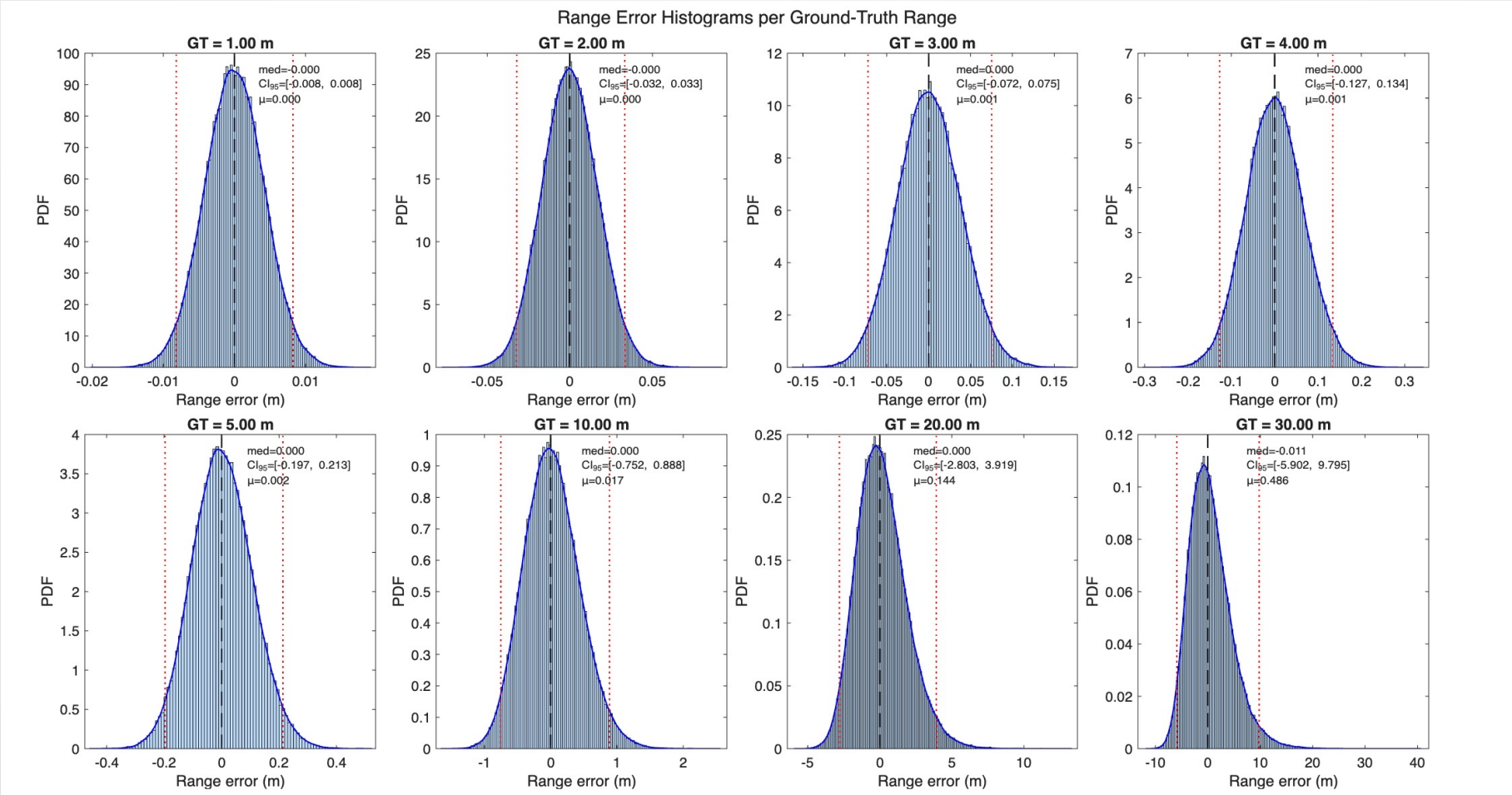
$$\mathbf{x}_s = (u_L, v_L, u_R)$$

- Depth uncertainty is encoded in the uncertainty of u_R

Close and Far Stereo Keypoints

- **Stereo keypoints are classed as *close* or *far* depending upon the distance**

Depth Error for Different Ground Truth Distances



Close and Far Stereo Keypoints

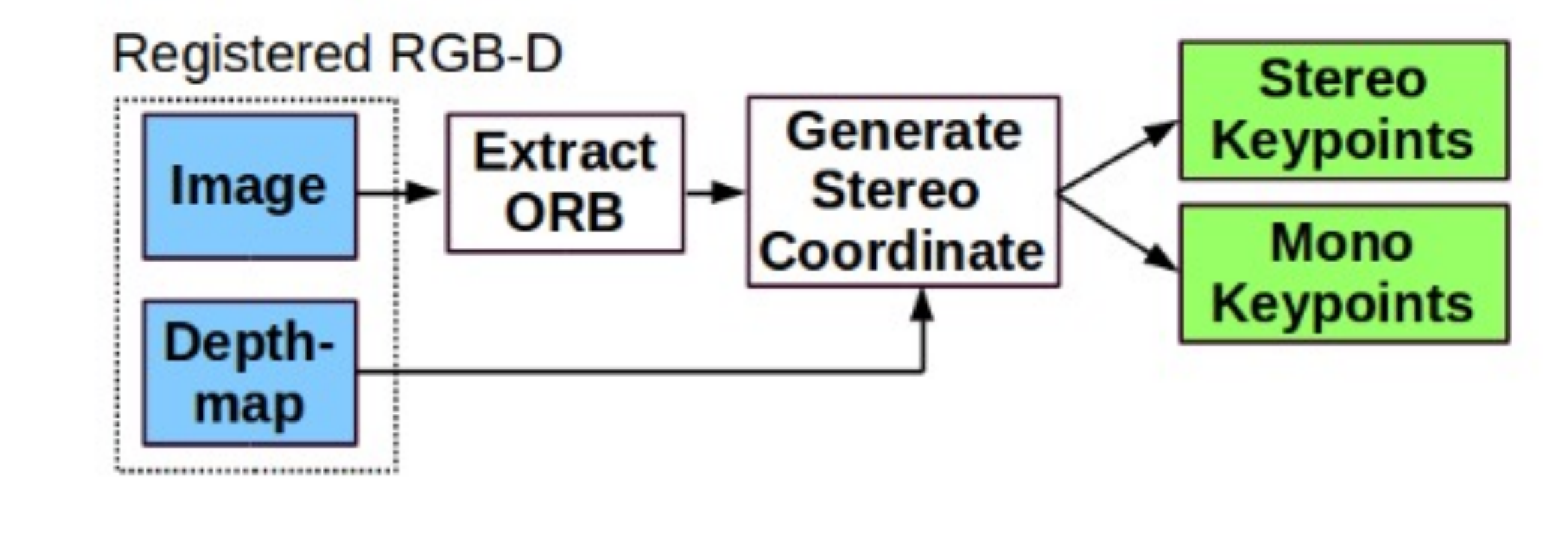
- Stereo keypoints are classed as *close* or *far* depending upon the distance
- Close keypoints are closer than 40 times the baseline and can be triangulated to compute depth
- Far keypoints are further than 40 times the baseline and are treated as monocular features

RGB-D Camera

- RGB-D cameras combine regular RGB data with a depth camera
- The depth camera uses structured light or time of flight to measure the depth to each pixel
- This produces a depth map

RealSense Camera

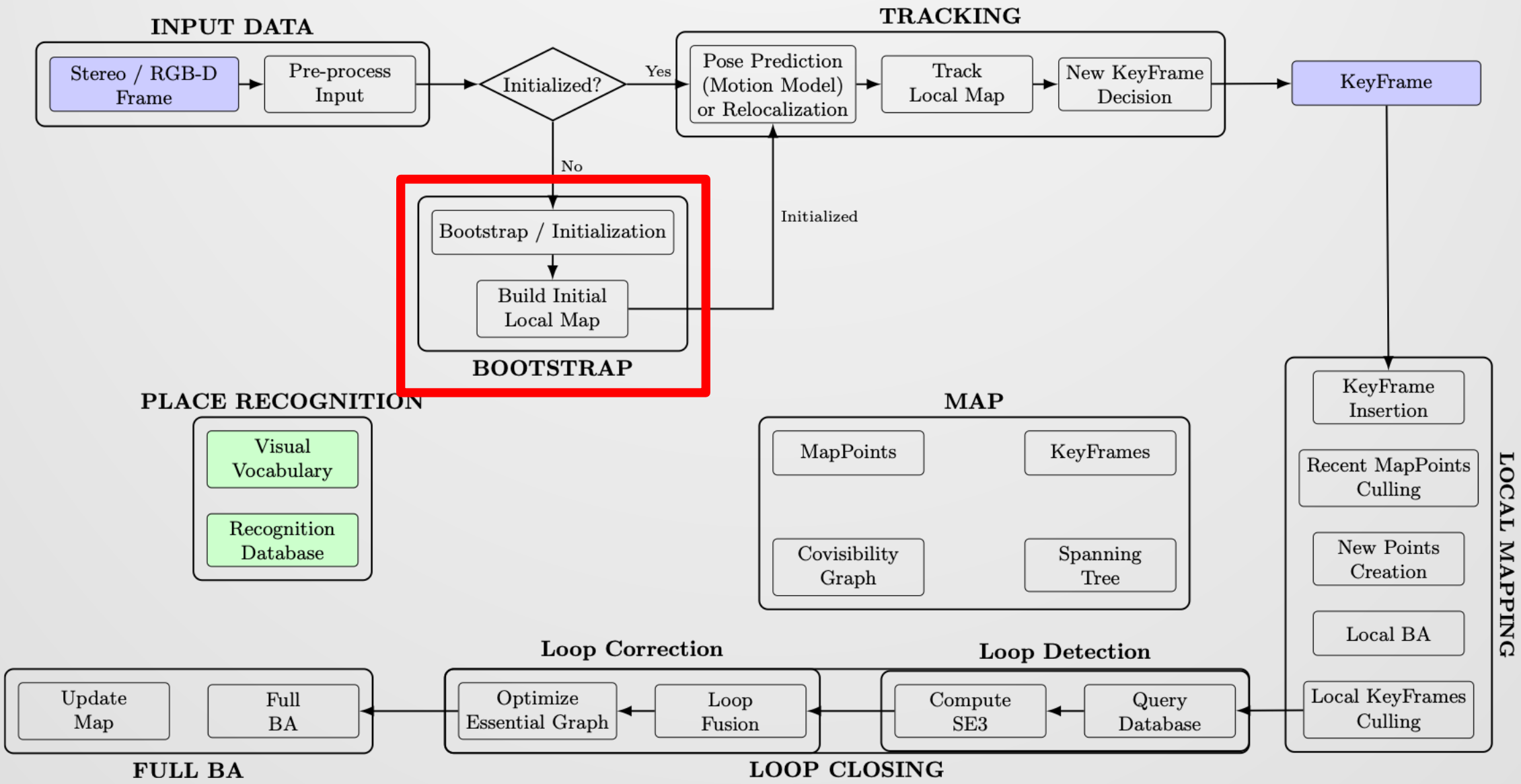
Pre-Processing the RGB-D Camera



RGB-D Camera

- ORB-SLAM is pretty hacky here – it pretends depth features are stereo features
- The ORB descriptors are extracted to give (u_L, v_L)
- The depth associated with each ORB feature is identified
- A synthetic stereo observation is formed, where

$$u_R = u_L - \frac{f_x b}{d}$$


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

System Bootstrapping

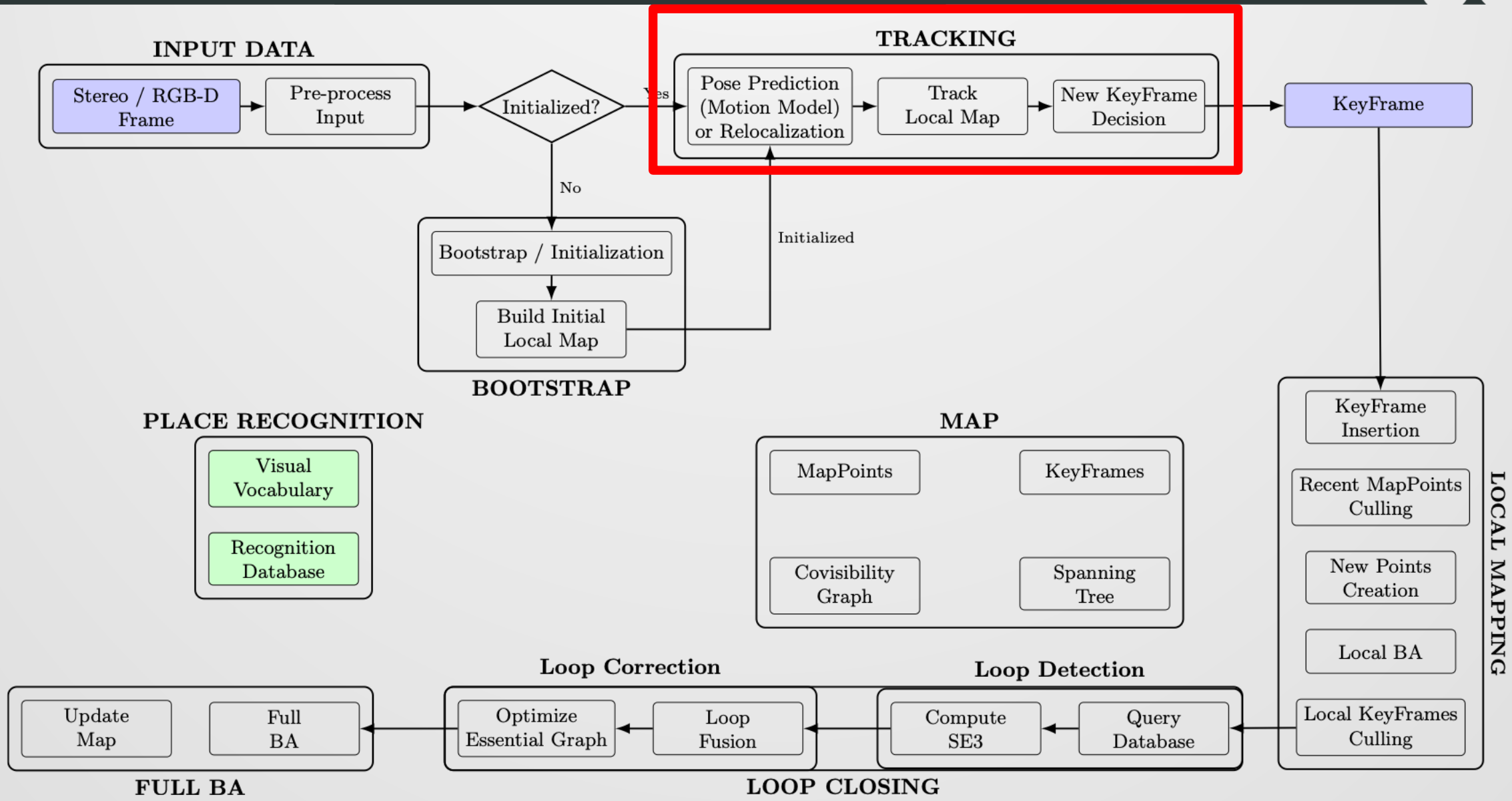
- Before ORB-SLAM2 can start, it has to bootstrap the initial frames
- Unlike COLMAP, it can’t build a scenegraph over all the images and then hunt for a start location
- Instead, it has to initialise from an input sequence

System Bootstrapping for Stereo / RGB-D

- Since these sensors directly measure depth this is easy
- As soon as you see a frame with enough features in it that you can initialize depth to, initialize the map points

System Bootstrapping for Monocular Systems

- Monocular systems can’t measure depth and so feature points have to be estimated by looking at the movement of points over multiple frames
- Feature-based tracking from frame to frame is used to match individual frames
- RANSAC is run with both fundamental matrices and homographies until a model can be successfully fit


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Bootstrap / Initialization
Build Initial Local Map
Initialized
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
LOCAL MAPPING
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

TRACKING
Yes
Pose Prediction
(Motion Model)
or Relocalization
Track
Local Map
New KeyFrame
Decision

Pose Prediction / Relocalization

- The purpose of this step is to estimate the next position and orientation of the camera
- Two possible options can be done:
  - If the system is currently tracking, predict pose from the next frame
  - If it’s not tracking, relocalize

Pose Prediction

- Since the frames always arrive sequentially in ORB-SLAM, the idea of being able to predict over time makes sense
- ORB-SLAM does not store a process model
- However, it attempts to predict camera movement over frames to aid feature matching
- A constant linear / angular velocity motion model is used
- Velocity is from estimated pose from previous two frames

Relocalization

- If the tracking system has become lost, ORB-SLAM does not reinitialize from scratch
- Rather, it uses the same procedure as loop closing to see if it can recognize where it is
- If it’s successful, the computed estimate is used
- We’ll look at loop closing later

TRACKING
Yes
Pose Prediction
(Motion Model)
or Relocalization
Track
Local Map
New KeyFrame
Decision

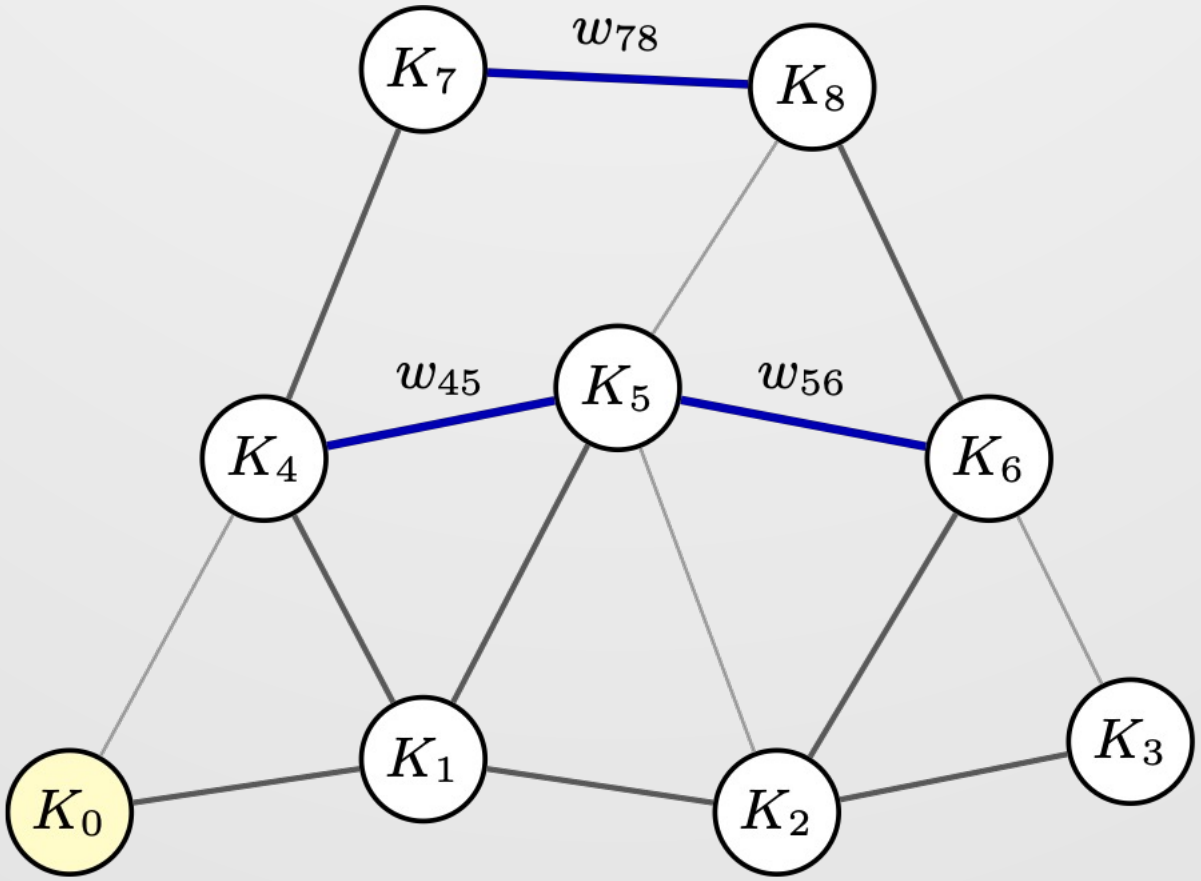
Track Local Map

- This is the *real time* tracking system present in ORB-SLAM
- The idea is to come up with the smallest optimization problem you can solve
- To do this, we need to understand a bit more how ORB-SLAM organizes its map

ORB-SLAM Map Management

- ORB-SLAM uses a dynamic sub-mapping strategy to identify features to match against
- At any time, there is an active map which contains two levels:
  - $\mathcal{K}_1$ is the set of all frames directly visible to the camera
  - $\mathcal{K}_2$ is the set of all frames co-visible to the frames in $\mathcal{K}_1$
- ORB-SLAM also identifies $K_{\mathrm{ref}} \in \mathcal{K}_1$, which is the frame which shares the most features with the currently visible frame

ORB-SLAM Local Map



ORB-SLAM Local Map Feature Matching

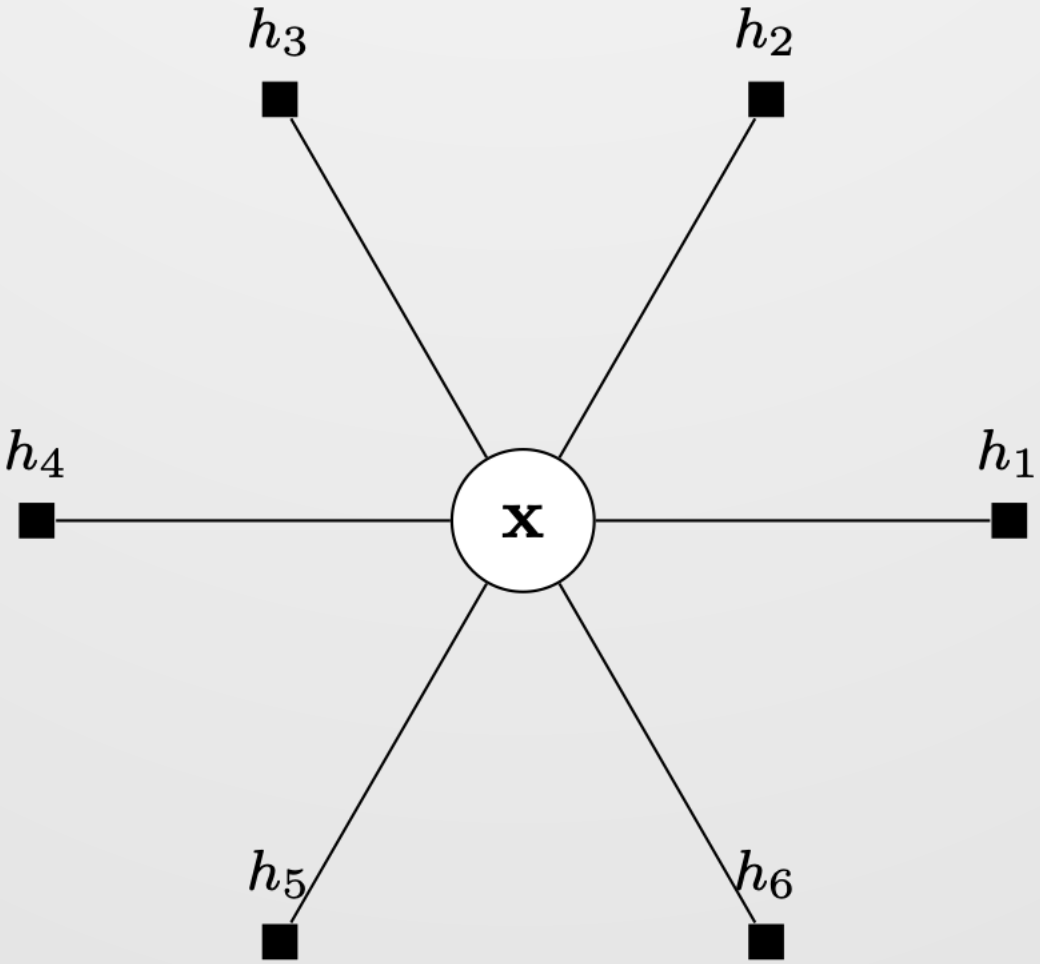
- Compute the projection of each key point into the frame

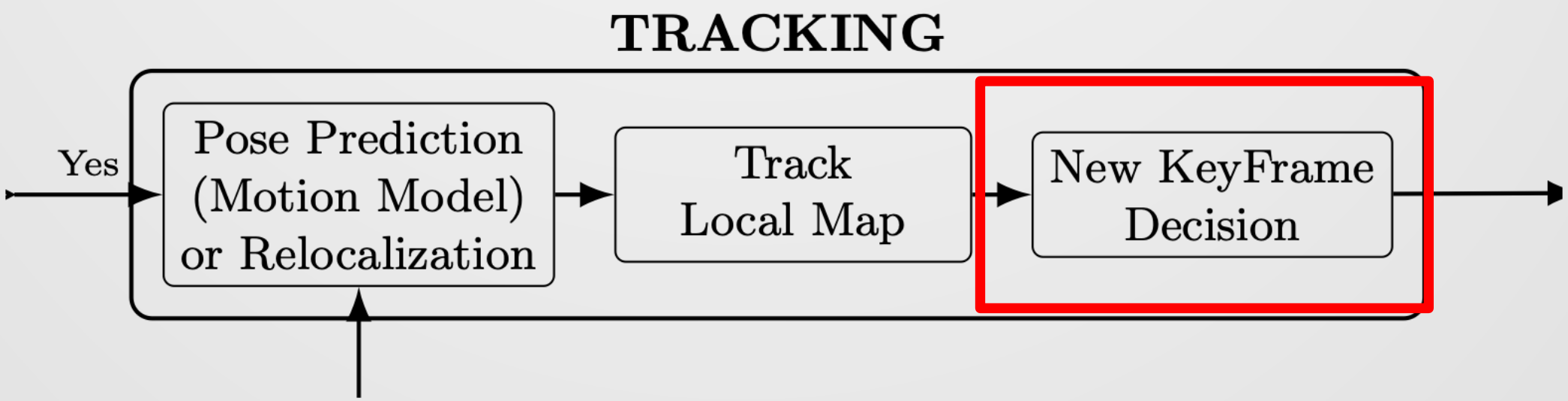
- Discard a keypoint if:
  - The angle between the ray from the camera centre and $\mathbf{n}_i$ is greater than 60 degrees
  - The distance to the point falls outside the range $d \in [d_{\mathrm{min}}, d_{\mathrm{max}}]$

Tracking Optimization Problem

- Pose optimization uses a small factor graph constructed in g2o
- The graph contains a *single* vertex which is the pose of the camera
- For each key point that's tracked an edge is created
  - The edge uses the perspective projection equation
  - The key point position is fixed
- The optimizer is run multiple times for robustness
  - Features can switch between being marked as inliers and outliers

Pose Optimizer



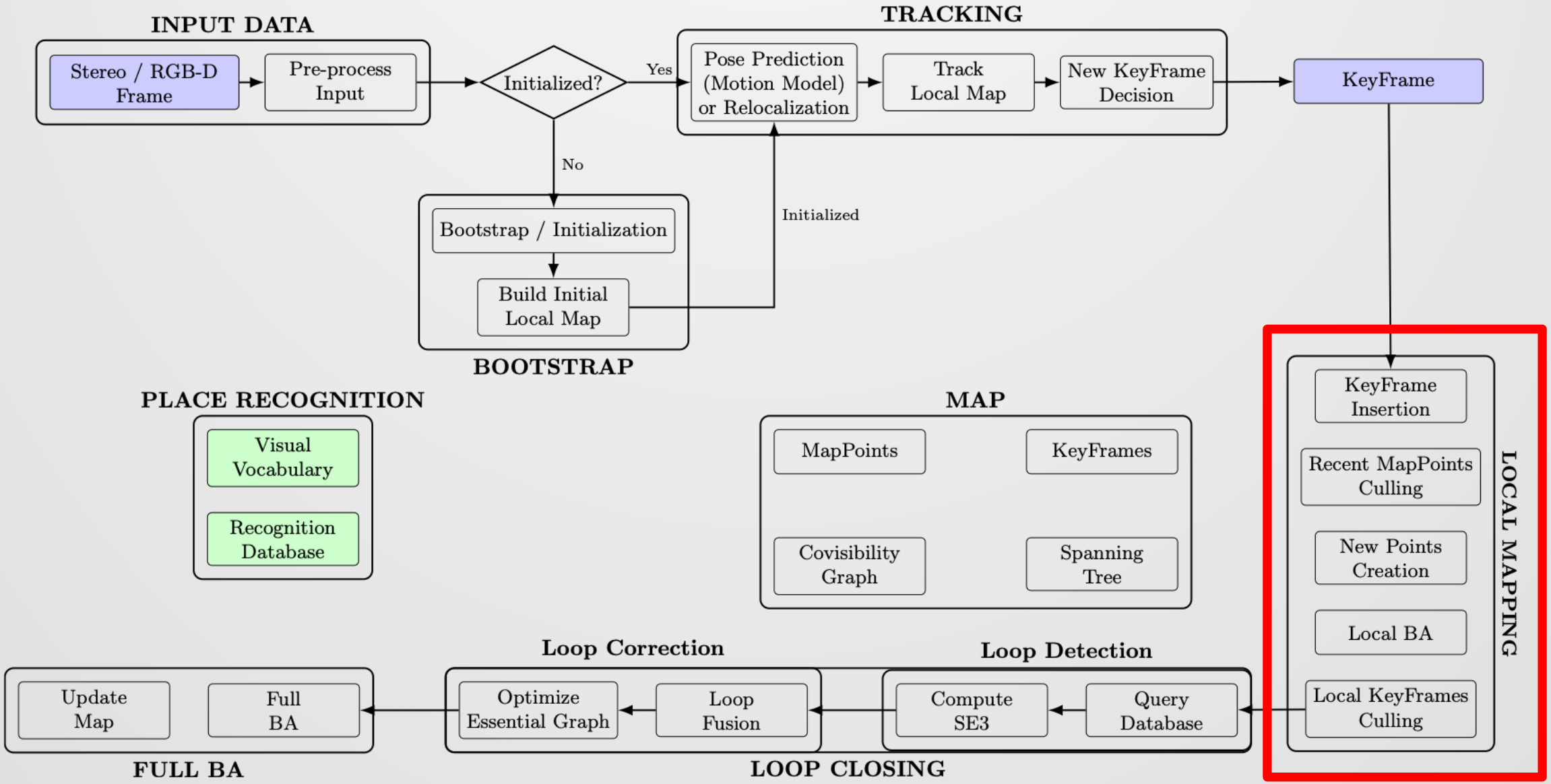

TRACKING
Yes
Pose Prediction
(Motion Model)
or Relocalization
Track
Local Map
New KeyFrame
Decision

New Key Frame Decision

- COLMAP treats all images as keyframes
- ORB-SLAM is more selective and only selects some frames:
  - This reduces the map size and hence computational cost / storage
  - Can increase robustness to some harsh movements and rotation
- Heuristics such as tracking quality and potential new information content are used

New Key Frame Tracking Decisions

- More than 20 frames since last global relocalization
  - Ensures the map has globally “settled down”
- Local mapping is idle, or more than 20 frames since last keyframe insertion
  - Ensures the map has locally “settled down”
- Current frame tracks at least 50 points
  - Good track with a good position estimate
- Current frame tracks less than 90% features in $\mathcal{K}_{\mathrm{ref}}$
  - Guarantees including new features not seen before


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING

Key Frame Insertion

- The new keyframe K_i is inserted into the co-visibility graph and the edge updated with the number of shared points
- K_i is inserted into the essential graph
- The bag-of-words representation for K_i is computed

Recent Map Points Culling

- Even though a point has been added, its survival is not guaranteed because there could be an error (wrong data association)
- Map points are monitored for 3 *key frames* after insertion:
  - It must be visible for more than 25% of the *frames* in which it is visible
  - If more than one keyframe has passed from map point creation, it must be observed from at least three keyframes

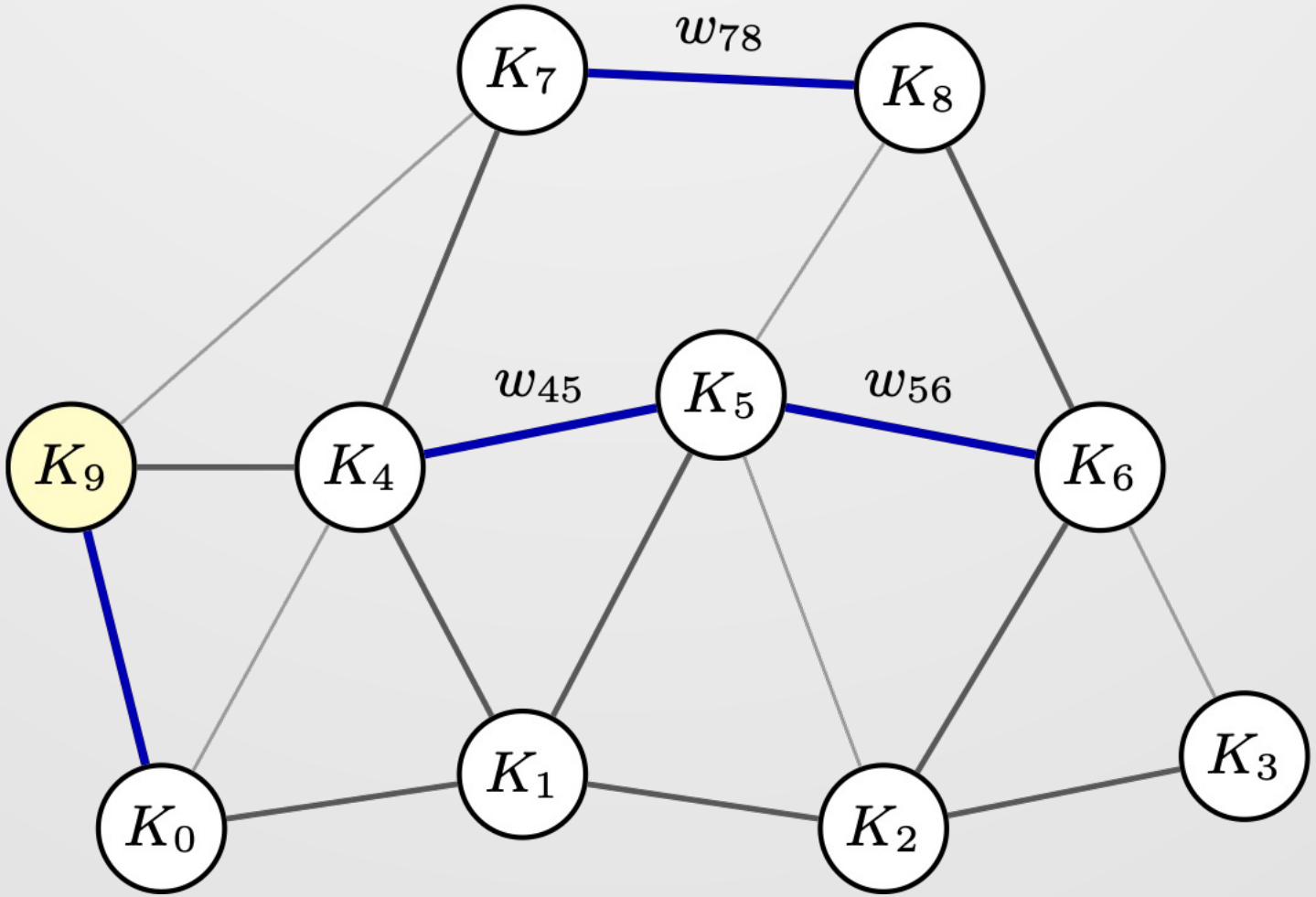
New Map Point Creation

- A new map point is created initially from two keyframes:
  - Feature descriptors have to match
- However, the map point could appear in more keyframes:
  - For example, in earlier frames the descriptors match but the baseline wasn't big enough
- To test for this, the co-visibility graph is checked and further relationships updated
  - Makes it possible to use geometric test over multiple frames

Local Bundle Adjustment

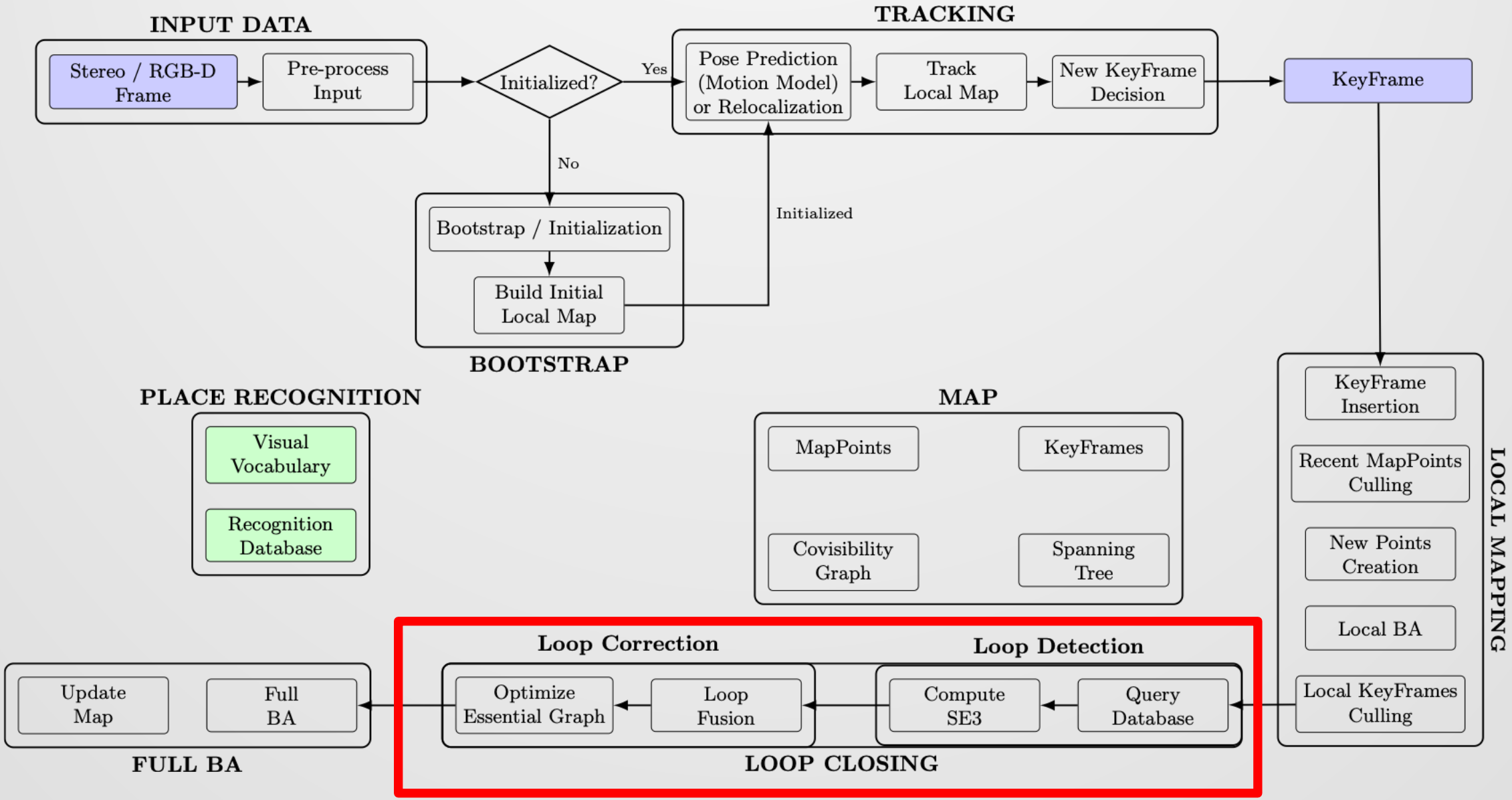
- This is used to optimize newly entered keyframes
- A factor graph is built which consists of:
  - The camera pose for the new keyframe
  - The cameras poses for all the keyframes connected to the new keyframe by the co-visibility graph
  - The map points contained in all these keyframes
- Multiple optimization rounds with outlier detection are used

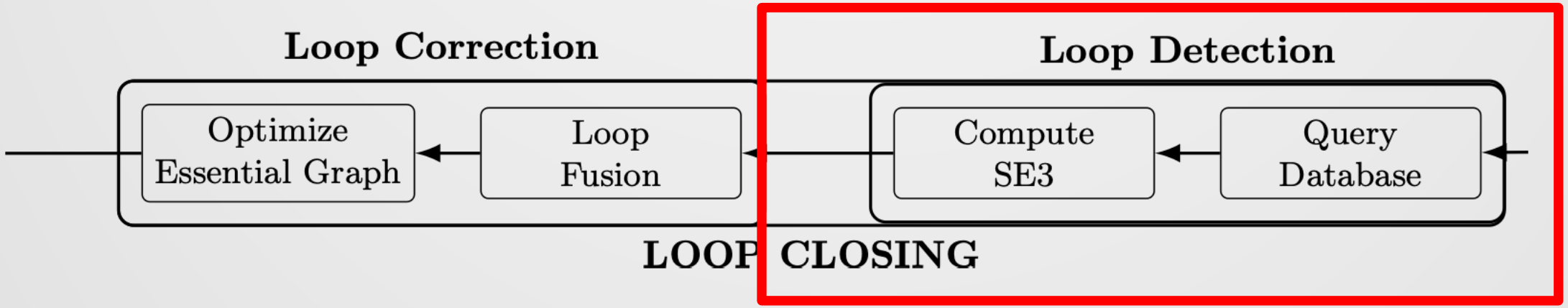
Local Bundle Adjustment



Local Keyframe Culling

- This is a further step to keep the complexity under control
- It identifies keyframes which appear to be redundant and remove then
- In a redundant keyframe, 90% of its map points have been seen in at least three other key frames at the same or finer scale of the ORB descriptors


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Correction
Loop Detection
Update Map
Full BA
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
FULL BA
LOOP CLOSING


Loop Correction
Loop Detection
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
LOOP CLOSING

Loop Detection and Threshold Calculation

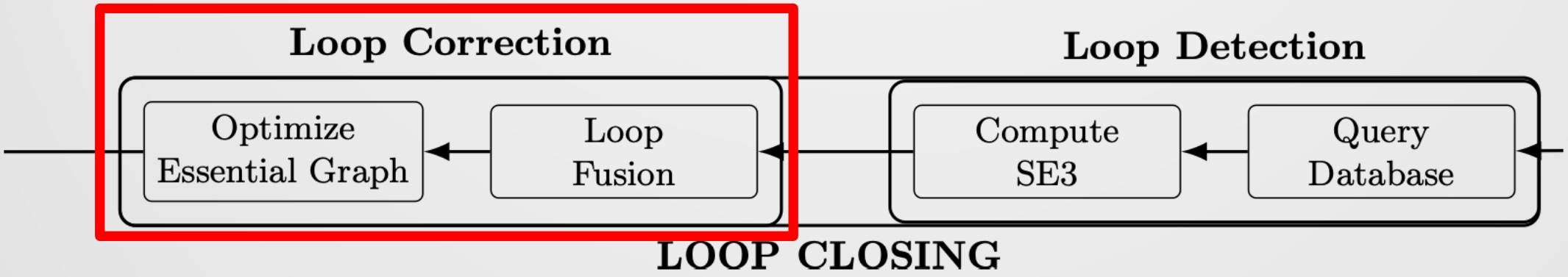
- This searches the map to see if the new keyframe K_i can be connected with earlier keyframes in the map
- We need to compute a baseline recognition score
- Therefore, the bag of words similarity is computed with all the neighbouring keyframes from the co-visibility graph
- The lowest score defines a lower bound threshold

Loop Detection

- All keyframes are searched and those which are sufficiently similar are marked as loop closure candidates
- If the same candidate is consecutively three times the similarity transformation is computed
- The similarity transformation is the geometric transformation needed to correct the alignment

Computing the Similarity Transformation

- This is the transformation required to complete loop closure
- When stereo or RGBD is used, this is six degrees of freedom
- When monocular is used, this is seven degrees of freedom
- The transformation is computed by checking the current keyframe with the loop candidate keyframe
- RANSAC is used with the appropriate geometric model


Loop Correction
Loop Detection
Optimize Essential Graph
Loop Fusion
Compute SE3
Query Database
LOOP CLOSING

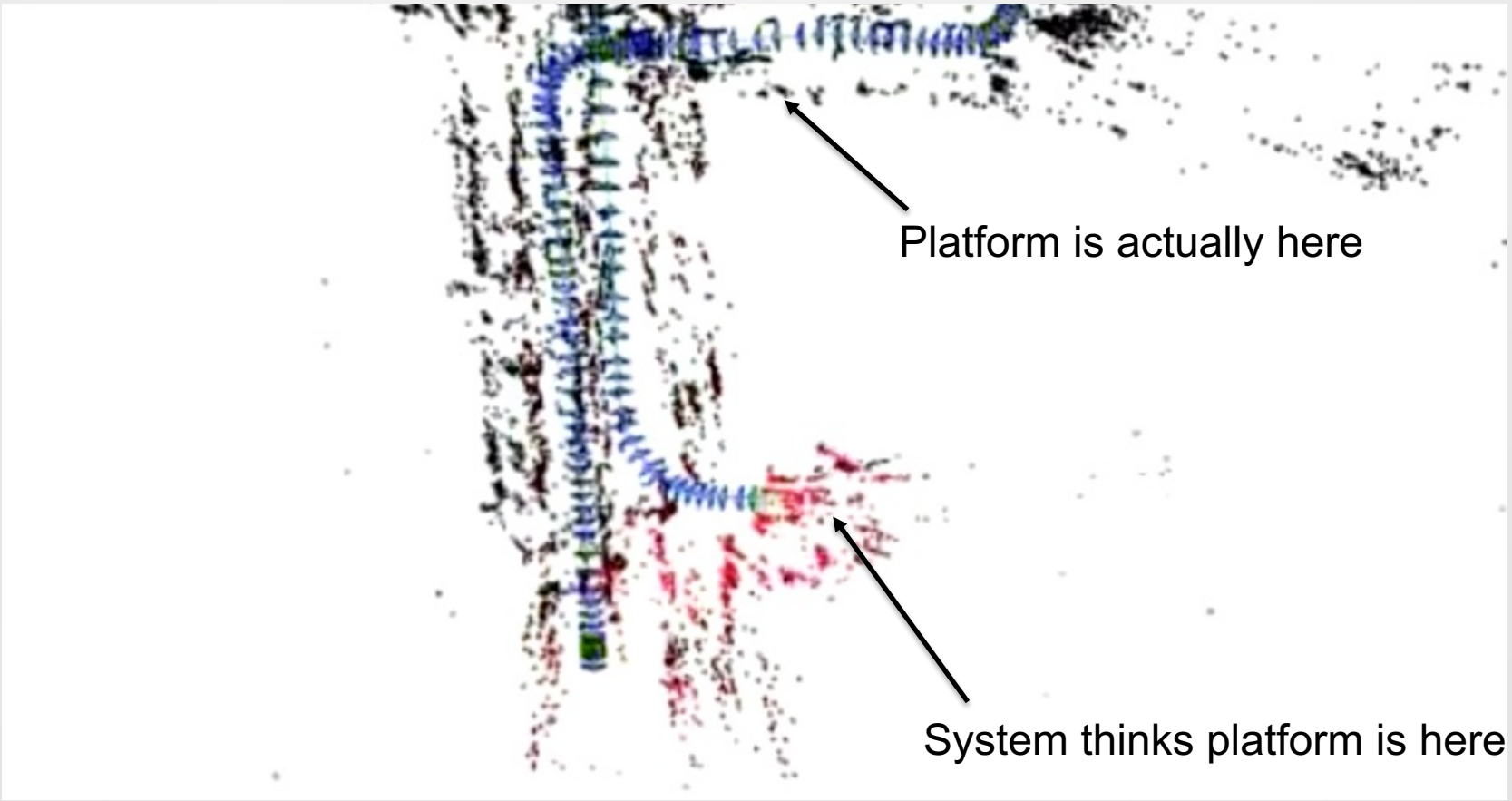
Loop Correction

- Factor graphs will automatically incorporate loop closure constraints
- However, we need to consider two issues:
  - Duplicate keypoints can be created
  - We don’t want to force the system to solve a giant optimization problem because it will “freeze” the system

Loop Fusion

- **Before loop closure, duplicate copies of map points are created**

Before Loop Fusion in the KITTI Dataset

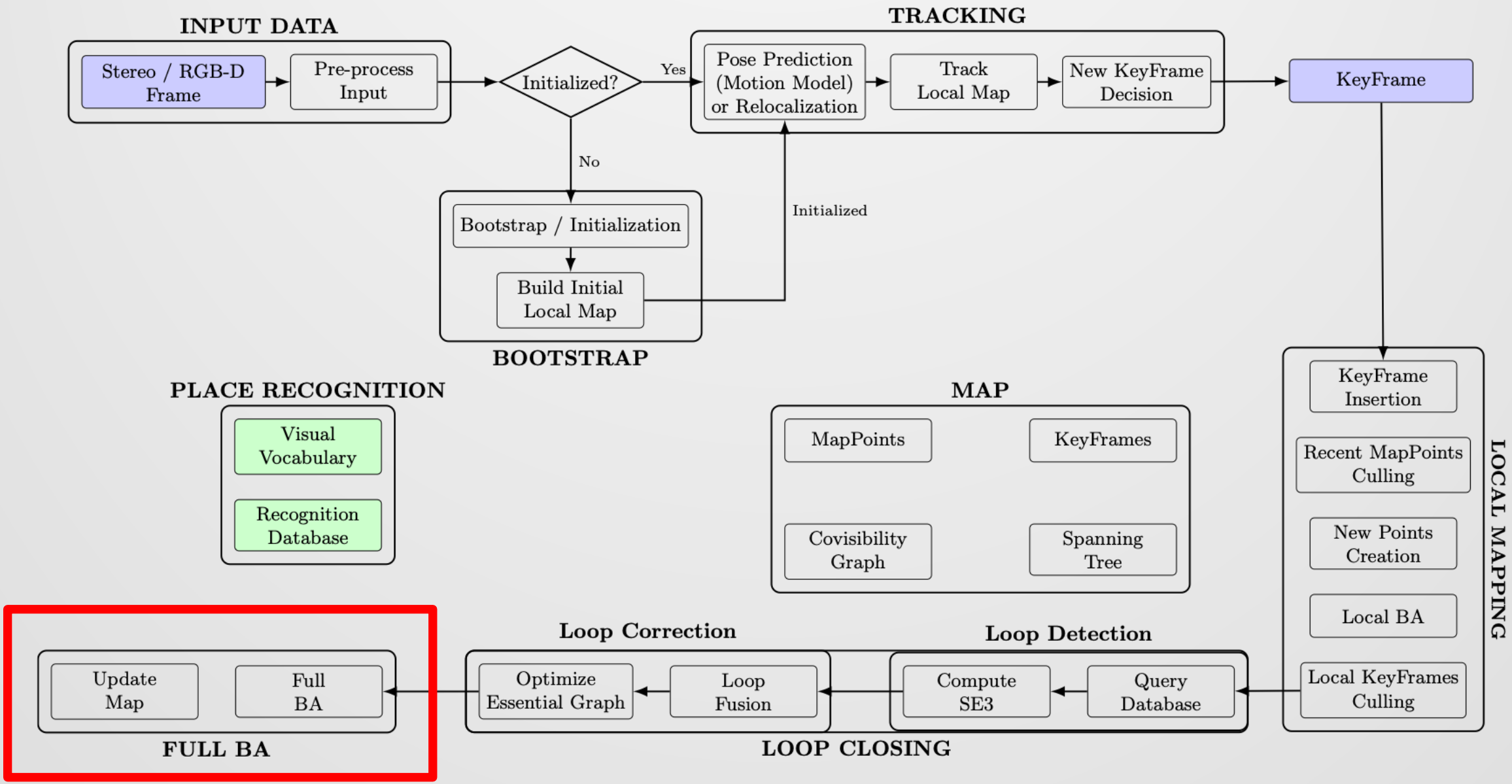


Loop Fusion

- Before loop closure, duplicate copies of map points are created
- In this step, the duplicates are removed
- The computed similarity transform is used to correct the keyframe locations
- The updated transformation shifts all the keypoints
- Updated keypoints which lie sufficiently close to one another are merged

Essential Graph Optimization

- We want a cheap and nasty way to incorporate loop closure without doing full BA
- A factor graph is built using the Essential Graph
  - This *only* contains the camera poses
- Once the graph has been optimized, the keypoints are shifted by the amount the keyframe pose has changed


INPUT DATA
Stereo / RGB-D Frame
Pre-process Input
Initialized?
Yes
No
TRACKING
Pose Prediction (Motion Model) or Relocalization
Track Local Map
New KeyFrame Decision
KeyFrame
Initialized
Bootstrap / Initialization
Build Initial Local Map
BOOTSTRAP
PLACE RECOGNITION
Visual Vocabulary
Recognition Database
MAP
MapPoints
KeyFrames
Covisibility Graph
Spanning Tree
LOCAL MAPPING
KeyFrame Insertion
Recent MapPoints Culling
New Points Creation
Local BA
Local KeyFrames Culling
Loop Detection
Compute SE3
Query Database
Loop Correction
Optimize Essential Graph
Loop Fusion
LOOP CLOSING
Update Map
Full BA
FULL BA

Full BA

- After optimizing the Essential Graph a Full Bundle Adjustment can be run
- This optimizes all of the keyframe and map point positions
- It can be time consuming, so it is run in a separate thread from everything else
- This makes sure that real-time operation is preserved

Full BA

- Full BA can continue to run as long as no map changing events (a new keyframe is inserted, a new loop is closed) happen
- If these events do happen, the optimizer terminates at the current step
- Since at each step of the optimizer the cost goes down, this means even intermediate values should be okay to use

Example

Kenilworth 3

Summary

- This completes our high-level overview of COLMAP and ORB-SLAM2
- Both systems start with factor graphs but use them in different ways
- Both systems exploit both the accuracy, flexibility and robustness that factor graphs provide

Where Next?

- After reading week, we'll look at how we evaluate SLAM systems
- We'll see how we can use COLMAP to evaluate the performance of ORB-SLAM2
- We'll then move on to looking at point clouds, as well as 2D and 3D occupancy grids